Experiment 1: Vector and Matrix Operations for Deep Learning

Aim

To perform vector and matrix operations using NumPy, calculate cosine similarity between customers, manually compute a dense-layer output, and visualise customer–book recommendation scores using a heatmap.

Mathematical Formulae

1. Dot product

$$a \cdot b = \sum_{i=1}^{n} a_i b_i$$

2. Matrix multiplication

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$$

3. Cosine similarity

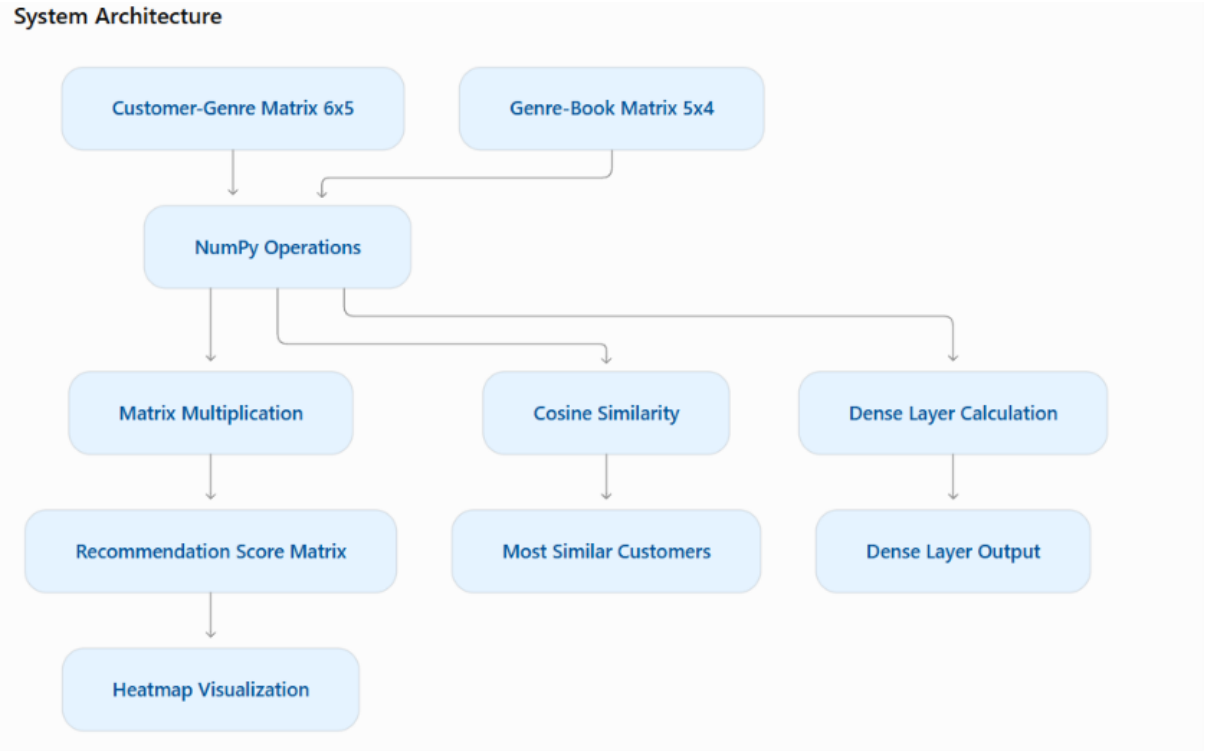
$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

4. Dense layer

$$y = Wx + b$$

5. Matrix transpose

$$(A^T)_{ij} = A_{ji}$$



Algorithm / Procedure

1. Import NumPy and Matplotlib.
2. Create the customer–genre and genre–book matrices.
3. Calculate the dot product, matrix multiplication, transpose and element-wise operations.
4. Calculate the inverse of a valid square matrix.
5. Compute cosine similarity between all customer pairs.
6. Identify the two most similar customers.
7. Calculate the dense-layer output using \(y=Wx+b\).
8. Plot the customer–book score matrix as a heatmap.

Python Program

```
import numpy as np

import matplotlib.pyplot as plt


# 1. Customer-genre matrix (6 customers, 5 genres)

customers = np.array([

  [5, 4, 1, 2, 3],
```

```
    [4, 5, 1, 2, 3],
    [1, 2, 5, 4, 1],
    [2, 1, 4, 5, 2],
    [3, 3, 2, 4, 5],
    [5, 4, 2, 1, 3]
], dtype=float)

# 2. Genre-book matrix (5 genres, 4 books)
books = np.array([
    [0.9, 0.2, 0.1, 0.4],
    [0.8, 0.3, 0.2, 0.5],
    [0.1, 0.9, 0.8, 0.2],
    [0.2, 0.8, 0.9, 0.3],
    [0.4, 0.2, 0.3, 0.9]
])

print("Customer Matrix:\n", customers)
print("\nBook Matrix:\n", books)

# 3. Dot product of two vectors
print("\nDot Product:",
      np.dot(customers[0], customers[1]))

# 4. Matrix multiplication: customer-book scores
scores = np.dot(customers, books)
print("\nCustomer-Book Score Matrix:\n", scores)

# 5. Transpose
print("\nTranspose of Customer Matrix:\n",
      customers.T)

# 6. Inverse (a square matrix is required)
square_matrix = np.array([
    [2, 1, 0],
```

```
    [1, 3, 1],
    [0, 1, 2]
], dtype=float)

print("\nInverse of Square Matrix:\n",
      np.linalg.inv(square_matrix))

# 7. Element-wise operations
print("\nAddition:\n", customers + customers)
print("\nMultiplication:\n", customers * customers)

# 8. Cosine similarity between customers
norms = np.linalg.norm(customers, axis=1)
similarity = (customers @ customers.T) / np.outer(norms, norms)

print("\nCosine Similarity Matrix:\n",
      np.round(similarity, 3))

# Find the most similar pair (excluding self-similarity)
upper = np.triu(similarity, k=1)
i, j = np.unravel_index(np.argmax(upper), upper.shape)

print(f"\nMost Similar Customers: Customer {i+1} and Customer {j+1}")
print("Similarity:", round(similarity[i, j], 3))

# 9. Dense layer: y = Wx + b
x = customers[0]  # First customer
W = books.T      # Shape: 4 x 5
b = np.array([0.1, 0.2, 0.1, 0.3])

y = W @ x + b

print("\nDense Layer Output for Customer 1:\n", y)
```

```
# 10. Heatmap of customer-book scores
plt.figure(figsize=(8, 5))
plt.imshow(scores, cmap="viridis", aspect="auto")
plt.colorbar(label="Recommendation Score")
plt.xticks(range(4), ["Book 1", "Book 2", "Book 3", "Book 4"])
plt.yticks(range(6), ["Customer 1", "Customer 2",
                      "Customer 3", "Customer 4",
                      "Customer 5", "Customer 6"])
plt.title("Customer-Book Recommendation Scores")
plt.xlabel("Books")
plt.ylabel("Customers")
plt.tight_layout()
plt.show()
```

Output

```
Customer Matrix:
 [[5. 4. 1. 2. 3.]
 [4. 5. 1. 2. 3.]
 [1. 2. 5. 4. 1.]
 [2. 1. 4. 5. 2.]
 [3. 3. 2. 4. 5.]
 [5. 4. 2. 1. 3.]]

Book Matrix:
 [[0.9 0.2 0.1 0.4]
 [0.8 0.3 0.2 0.5]
 [0.1 0.9 0.8 0.2]
 [0.2 0.8 0.9 0.3]
 [0.4 0.2 0.3 0.9]]

Dot Product: 54.0

Customer-Book Score Matrix:
 [[9.4 5.3 4.8 7.5]
 [9.3 5.4 4.9 7.6]
 [4.2 8.7 8.4 4.5]
 [4.8 8.7 8.7 5.4]
 [8.1 7.5 7.6 8.8]
 [9.3 5.4 4.7 7.4]]

Transpose of Customer Matrix:
 [[5. 4. 1. 2. 3. 5.]
 [4. 5. 2. 1. 3. 4.]
 [1. 1. 5. 4. 2. 2.]
 [2. 2. 4. 5. 4. 1.]
 [3. 3. 1. 2. 5. 3.]]

Inverse of Square Matrix:
 [[ 0.625 -0.25   0.125]
 [-0.25   0.5   -0.25 ]
 [ 0.125 -0.25   0.625]]

Addition:
 [[10.  8.  2.  4.  6.]
 [ 8. 10.  2.  4.  6.]
 [ 2.  4. 10.  8.  2.]
 [ 4.  2.  8. 10.  4.]
 [ 6.  6.  4.  8. 10.]
 [10.  8.  4.  2.  6.]]
```

```
Multiplication:
 [[25. 16.  1.  4.  9.]
 [16. 25.  1.  4.  9.]
 [ 1.  4. 25. 16.  1.]
 [ 4.  1. 16. 25.  4.]
 [ 9.  9.  4. 16. 25.]
 [25. 16.  4.  1.  9.]]

Cosine Similarity Matrix:
 [[1.    0.982 0.57  0.648 0.883 0.982]
 [0.982 1.    0.59  0.629 0.883 0.964]
 [0.57  0.59  1.    0.949 0.735 0.59 ]
 [0.648 0.629 0.949 1.    0.837 0.629]
 [0.883 0.883 0.735 0.837 1.    0.849]
 [0.982 0.964 0.59  0.629 0.849 1.   ]]

Most Similar Customers: Customer 1 and Customer 2
Similarity: 0.982

Dense Layer Output for Customer 1:
 [9.5 5.5 4.9 7.8]
```



Result

Thus, vector and matrix operations were performed using NumPy. Cosine similarity was used to identify similar customers, the dense-layer equation was evaluated manually, and a heatmap was generated to visualise recommendation scores.